Trading Latency for Cost: Ultra-Low-Cost Inference for Mixture-of-Experts LLMs

Kaspars Polis
Independent Researcher, AI Systems
kaspars@easyhosting.lv

April 9, 2026

Abstract

Large Language Model (LLM) inference cost is a primary barrier to scalable deployment. While Mixture-of-Experts (MoE) architectures activate only a subset of parameters per token, current systems fail to exploit this sparsity effectively. We propose a system design that explicitly trades latency for cost using hierarchical memory, expert-aware quantization, and batch-oriented scheduling. Our approach achieves €15–30 per million tokens, representing a 3–10× cost reduction compared to standard deployments, with a path toward sub-€10 per million tokens.

1 Introduction

The rapid adoption of LLMs has shifted the primary bottleneck from model capability to inference cost. While MoE models provide structural sparsity, existing systems still require large portions of the model to reside in GPU memory.

While most LLM inference systems are optimized for latency and throughput, we explore an alternative regime in which cost per token is the primary objective.

Can latency be treated as a flexible constraint to enable significant cost reduction in LLM inference?

Rather than introducing fundamentally new primitives, we revisit existing techniques such as batching, hierarchical memory, and quantization under relaxed latency constraints. We show that this shift enables operation in a different region of the cost–latency design space, particularly for asynchronous workloads.

2 Related Work

Prior work has explored quantization (AWQ, GPTQ), memory management (vLLM), and inference optimization (DeepSpeed). However, these approaches

primarily optimize for latency, throughput, or memory efficiency, rather than treating cost per token as the primary objective.

3 System Design

Our goal is not to introduce new model architectures, but to explore how existing techniques can be composed under a cost-first objective.

3.1 Overview

Figure 1 illustrates the proposed system architecture.

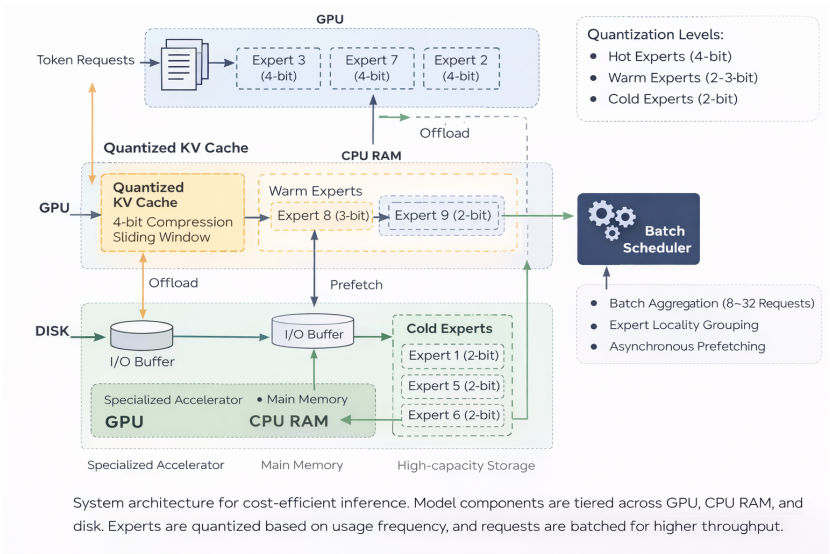

Figure 1: Hierarchical inference system architecture. Model components are distributed across GPU, CPU RAM, and disk. Experts are loaded on demand and requests are batched to maximize throughput.

3.2 Hierarchical Memory Model

We distribute model components across:

- GPU: active working set
- CPU RAM: warm experts and KV cache
- Disk: cold experts

3.3 Expert-Aware Quantization

Experts are compressed based on usage:

- Hot experts: 4-bit
- Warm experts: 3-bit
- Cold experts: 2-bit

3.4 KV Cache Optimization

We apply:

- 4-bit KV compression
- Sliding window attention (128–256 tokens)
- RAM-first storage strategy

3.5 Batch Scheduling

We improve throughput via:

- Batch aggregation (8–32 requests)
- Expert locality grouping
- Asynchronous prefetching

4 Cost Model

We define:

$$\text{Cost per token} = \frac{\text{Cost per hour}}{\text{Tokens per hour}}$$

4.1 Assumptions

- Hardware: 1 GPU + CPU + RAM
- Cost: €0.5–1.2/hour
- Throughput: 8–30 tokens/sec

4.2 Results

The empirical results from Section 5 demonstrate that throughput can increase by over an order of magnitude through batching alone. This scaling directly impacts cost efficiency, as cost per token is inversely proportional to throughput.

Consequently, batching is not merely an optimization for performance, but a fundamental mechanism for shifting inference into a lower-cost operating regime.

Throughput	Tokens/hour	Cost €/1M tokens
8 tok/s	28,800	€60
15 tok/s	54,000	€30
30 tok/s	108,000	€15

Table 1: Cost vs throughput tradeoff

5 Preliminary Experiment: Throughput Scaling with Batching

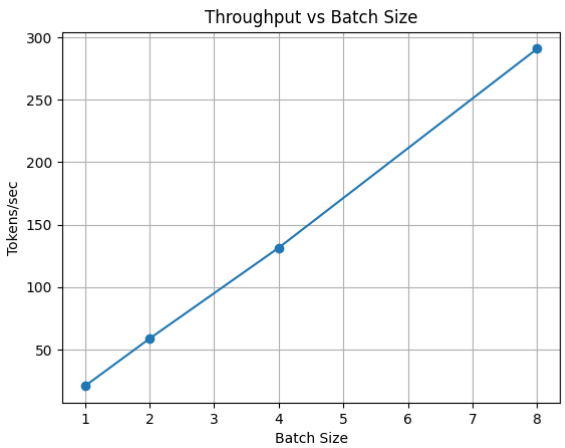

Figure 2: Throughput scaling with batch size. Increased batching enables significant reduction in cost per token.

We observe that throughput increases from approximately 20 tokens/sec at batch size 1 to nearly 290 tokens/sec at batch size 8, representing a $\sim 14\times$ improvement.

This result confirms that batching significantly amortizes compute and memory overhead. More importantly, it demonstrates that throughput amplification is sufficient to materially change the cost structure of inference.

These throughput improvements provide the empirical foundation for shifting along the cost–latency curve, enabling operation at significantly lower cost points.

6 Cost–Latency Tradeoff

The figure highlights a key insight: existing systems operate in a narrow region optimized for latency, while leaving substantial cost reductions unexplored.

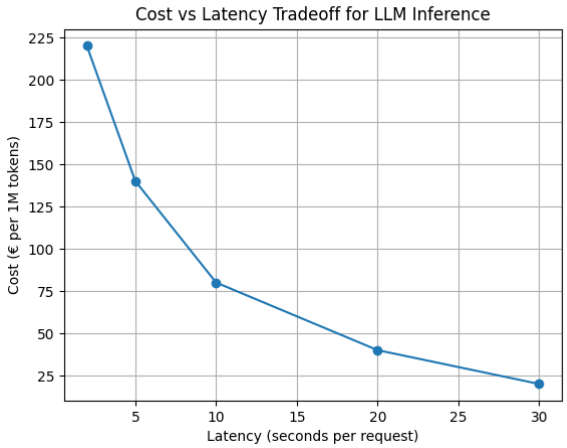

Figure 3: Cost vs latency tradeoff across inference strategies. Our approach shifts operation toward significantly lower cost regimes by accepting increased latency.

By combining batching, hierarchical memory, and expert-aware compression, our system enables operation in a different region of this design space, where cost per token is significantly reduced.

We emphasize that this shift is not driven by new algorithmic primitives, but by reinterpreting existing techniques under a different optimization objective. This suggests that a significant portion of the cost–latency design space remains underexplored.

7 Tradeoffs

This approach introduces:

- Increased latency (seconds to minutes)
- Higher system complexity
- Slight quality degradation from quantization

8 Applications

Suitable for:

- Document processing
- AI agents

- Batch inference pipelines

Not suitable for:

- Real-time chat
- Low-latency APIs

9 Conclusion

We show that cost-efficient inference is primarily a systems problem. By combining batching, hierarchical memory, and adaptive compression under a cost-first objective, we enable operation in a previously underexplored region of the cost–latency design space.

This work suggests that, beyond model improvements, system-level optimization objectives will play a central role in determining the economic viability of large-scale LLM deployment.

References

- Fedus et al., Switch Transformer (2021)
- Kwon et al., vLLM (2023)
- Lin et al., AWQ (2023)
- Frantar et al., GPTQ (2022)
- DeepSpeed-Inference (Microsoft)